

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Модуль BeautifulSoup

[BeautifulSoup](#) - это библиотека для синтаксического разбора файлов HTML/XML. Проще говоря, это библиотека, которая позволяет легко собирать информацию с веб-страниц.

Для установки этой библиотеки, нужно ввести следующую команду в терминале - `pip install beautifulsoup4`. Также её можно установить в линукс из репозитория командой `apt-get install python3-bs4`.

Объект `BeautifulSoup()` может принимать два аргумента. Первым аргументом является фактическая разметка, а второй аргумент – парсер или синтаксический анализатор HTML/XML. Анализаторы могут применяться разные, например `html.parser`, `lxml`, `html5lib`.

`html.parser` – это встроенный парсер

`lxml` – один из самых быстрых парсеров, установить можно командой в терминале `pip install lxml`

`html5lib` – медленный, установить можно командой в терминале `pip install html5lib`

Парсим HTML теги

Напишем простенький код:

```
import requests
from bs4 import BeautifulSoup

def get_page_data(html):
    soup = BeautifulSoup(html.text, 'lxml')
    print(soup)

if __name__ == '__main__':
    url = 'https://codeby.net/'
    page = requests.get(url)
    print(page.status_code)
    get_page_data(page)
```

Метод `requests.get()` возвращает объект `Response`. Проверяем успешность запроса с помощью `status_code`, и передаём объект в функцию `get_page_data` в которой уже создаётся объект `BeautifulSoup`. Этот объект принимает в качестве аргумента объект `Response` с применённым к нему свойством `text`. Вторым аргументом выступает парсер `lxml`.

Далее мы выводим на печать содержимое объекта, в котором содержится исходный HTML-код.

```
soup x
200
<!DOCTYPE html>
<html class="has-no-js template-EWRporta_articles_index uix_page--fluid
<head>
<meta charset="utf-8"/>
<meta content="IE=Edge" http-equiv="X-UA-Compatible"/>
<meta content="width=device-width, initial-scale=1, viewport-fit=cover"
<title>Форум информационной безопасности - Codeby.net</title>
<meta content="Программирование, этичный хакинг и тестирование на проник
```

Задача парсера вытащить определённую информацию со страниц сайта или документов. Сделаем простой поиск мета-тегов, например `title` и `H1`.

Ter title — это заголовок сайта, отражающий суть ресурса.

Ter H1 — это заголовок текстовой части страницы первого уровня.

```
import requests
from bs4 import BeautifulSoup

def get_page_data(html):
    soup = BeautifulSoup(html.text, 'lxml')
    print(soup.title.string)
    print(soup.h1.string)

if __name__ == '__main__':
    url = 'https://codeby.net/'
    page = requests.get(url)
    get_page_data(page)
```

```
soup x
Форум информационной безопасности - Codeby.net
Форум информационной безопасности - Codeby.net
```

Применив к объекту `soup` `title` и `h1` мы получаем теги вместе с их содержимым, а свойство `string` убирает сами теги, оставляя только строчное содержимое. Как мы видим, данные теги на ресурсе оказались совершенно одинаковые.

Теги можно также получить используя метод `select`, принимающий в качестве параметра теги, а также селекторы CSS – классы (`class`) и идентификаторы (`id`). Вывод результата будет в виде списка. Если мы хотим получить текст полученных элементов, то нужно воспользоваться методом `getText()`. Нужно учитывать, что данный метод применяется только к строкам, поэтому можно взять например элемент по индексу.

```

import requests
from bs4 import BeautifulSoup

def get_page_data(html):
    soup = BeautifulSoup(html.text, 'lxml')
    print(soup.select('title'))
    print(soup.select('h1')[0].getText())

if __name__ == '__main__':
    url = 'https://codeby.net/'
    page = requests.get(url)
    get_page_data(page)

```

soup x

```

[<title>Форум информационной безопасности - Codeby.net</title>]
Форум информационной безопасности - Codeby.net

```

Вместо `getText()` можно применить `get_text()` с аналогичным результатом.

С помощью свойства `attrs` можно получить атрибуты выбранного элемента. Запись `print(soup.select('h1')[0].attrs)` выдаст результат в виде словаря, содержащего название атрибута и его значение `{'class': ['p-title-value', 'to-guest-style']}`.

Поиск элементов по id и метод `find_all()`

Теперь поищем что-нибудь поинтереснее, например вытащим списки прокси-серверов со страницы https://top-proxies.ru/free_proxy.php.

The screenshot shows a web browser displaying a table of proxies. The table has two columns: 'IP:PORT' and an empty column. The first row contains the IP address '103.105.237.249:'. A context menu is open over the first row, showing options like 'Назад', 'Вперед', 'Перезагрузить', 'Сохранить как...', 'Печать...', 'Трансляция...', 'Перевести на русский', 'Просмотр кода страницы', 'Просмотр кода фрейма', 'Перезагрузить фрейм', and 'Просмотреть код'. The 'Просмотреть код' option is highlighted. The DOM tree on the right shows the HTML structure, with the table element and its tbody highlighted.

Любой поиск нужно начинать с исходного кода страницы. Исходник открывается нажатием на клавиатуре кнопки **F12**. А чтобы было совсем просто найти нужный элемент, то достаточно на нём щёлкнуть правой клавишей мышки и выбрать «Просмотреть код».

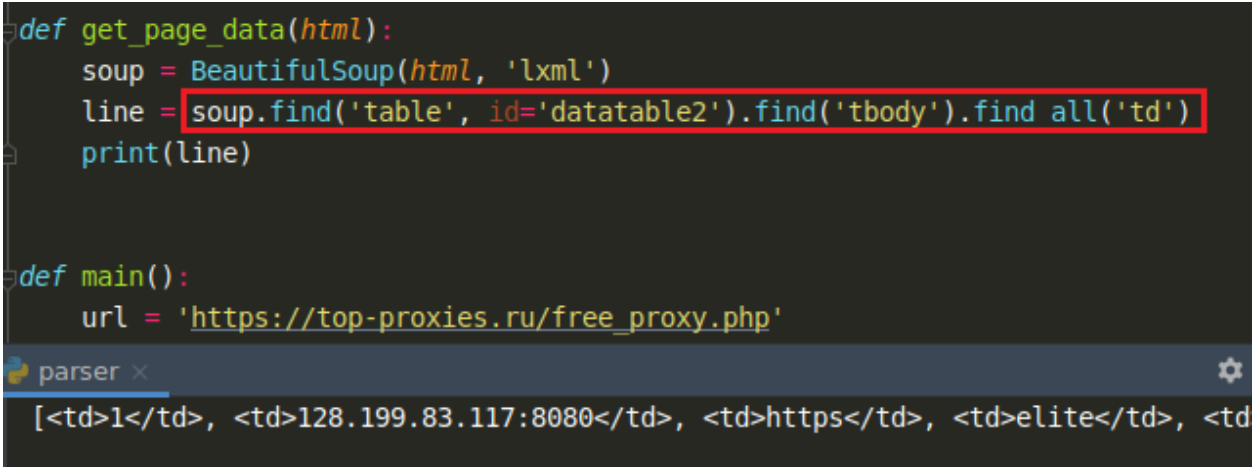
Мы видим что данные там расположены в таблице, имеющей id, об этом говорит эта строка - `table id="datatable2"`. Далее по коду видно что адрес прокси находится в теге `td`. Общий синтаксис таблиц следующий:

```
<table>
  <tr>
    <td>...</td>
  </tr>
</table>
```

Между открывающим тегом `<table>` и закрывающим тегом `</table>` находится тело таблицы. Тело состоит из строк и столбцов. Таблица заполняется построчно, где каждый тег `<tr>` создает новую строку. Далее во вложенных `<td>` создаются столбцы. Также существует тег `<tbody>`, который предназначен для хранения одной или нескольких строк таблицы. Допускается применять несколько тегов `<tbody>` внутри `<table>`.

Исходя из этого нам необходимо сначала найти тег `table` с `id="datatable2"`. И затем в найденной таблице выполнить поиск столбцов по тегу `<td>`, находящихся внутри тега `<tbody>`. Для одиночного поиска применяется метод `find`, а если нужно найти несколько элементов, то метод `find_all`.

Применим эту цепочку и получим строку с данными.



```
def get_page_data(html):
    soup = BeautifulSoup(html, 'lxml')
    line = soup.find('table', id='datatable2').find('tbody').find_all('td')
    print(line)

def main():
    url = 'https://top-proxies.ru/free_proxy.php'
```

parser x [<td>1</td>, <td>128.199.83.117:8080</td>, <td>https</td>, <td>elite</td>, <td>

Чтобы данные выглядели читаемо, прогоним их через цикл, применим свойство `text` для перевода в строку, и поставим условие поиска двоеточия, чтобы отсеять всё ненужное.

В итоге мы успешно спарсили более 1500 адресов прокси-серверов. Они довольно часто меняются, и итоговое количество выходит разное.

```
def get_page_data(html):
    soup = BeautifulSoup(html, 'lxml')
    line = soup.find('table', id='datatable2').find('tbody').find_all('td')
    print(line)

    for ip in line:
        if ':' in ip.text:
            print(ip.text)
```

parser ×

```
183.81.152.82:4145
103.10.59.73:4145
18.140.98.117:3128
182.253.21.26:32314
177.130.49.201:6699
159.224.37.181:60240
```

Того же результата мы можем достигнуть другим способом. Сначала мы найдём в таблице строки по тегу <tr>, и в строках уже далее ищем столбцы по тегу <td> в цикле. Таким образом, мы сможем взять элемент по индексу, и вывести его на печать.

```
def get_page_data(html):
    soup = BeautifulSoup(html, 'lxml')
    line = soup.find('table', id='datatable2').find('tbody').find_all('tr')

    for ip in line:
        td = ip.find_all('td')
        ip = td[1].text
        print(ip)
```

parser ×

```
188.212.102.223:5836
109.162.240.211:1085
180.210.222.237:1080
188.64.160.201:3128
117.254.61.133:43233
1.20.102.73:61919
```

Если нам нужно получить прокси только определённого типа, например socks5, то просто добавим дополнительное условие.

Соответствующие индексы идут в порядке очереди по столбцам слева-направо.

№	IP:PORT	ТИП	АНОНИМНОСТЬ	СТРАНА
1	179.106.170.241:47488	socks4	elite	BR
2	151.106.9.187:5836	https	transparent	FR

```
for ip in line:
    td = ip.find_all('td')
    ip = td[1].text
    if td[2].text == 'socks5':
        print(ip)
```

parser x

103.216.82.200:6667
121.230.231.35:38801
174.76.48.249:4145
174.70.241.14:24392
117.60.236.29:38801
104.248.63.49:30588